Dynamics Queue Admin

System & Architecture Design Document — dispatch-queue management console (`queadmin-frontend`, SPA), version 1.0.0

Document	**SDD-1.0.0** — System & Architecture Design Document
Application	Dynamics Queue Admin (*Queue Admin*) — dispatch-queue management console of the Dynamics freight marketplace
Asset type	Web Application
Asset code	**ASSET-DYN-QA-001**
Standard	INSA Secure Website Management Standard, Version 1.0, 2014 EC
Platform	Vite 8 + React 19 + TypeScript SPA (React Router 7), Redux Toolkit / RTK Query, socket.io-client 4, hand-written CSS
Production host	`queue.dynamicsroute.tech`
Design version	1.0.0 (as-built)
Companion documents	SRS, Technical Specifications (TDD/LLD), User Documentation (User Guide), Security Documentation
Submitting entity	**TBD**
Contact person	**TBD**
Prepared by	**TBD**
Reviewed by	**TBD**
Signatories	**TBD**
Date of submission	**TBD**

As-built document

This document describes the design of the submitted release of the Dynamics Queue Admin console as implemented in `queadmin-frontend`. It is the design counterpart to the SRS and is prepared for INSA Secure Website Management Standard review. Where behaviour is server-enforced in the backend (`transportBackEndNative`) it is stated explicitly, so reviewers can distinguish client responsibility from backend authority.

Contents

1	Introduction
2	System overview and context
3	Architectural design
4	Functional decomposition
5	Component design
6	Data design
7	Interface design
8	Security design
9	Deployment and hosting
10	Cross-cutting design
11	Design decisions and trade-offs
12	Requirements-to-design traceability
13	Known limitations and future work

List of Figures

2.1	Level 0 — Context diagram (external ecosystem)
3.1	Level 2 — Architecture layers
4.1	Level 1 — Functional decomposition

1. Introduction

1.1 Purpose

This document describes the architecture and internal design of the Dynamics Queue Admin web console (`queadmin-frontend`, SPA) as implemented. It defines the system context, the layered architecture, the functional decomposition, the components, data, interfaces, security posture, and deployment model of the console, and supports INSA submission review against the Secure Website Management Standard (Version 1.0, 2014 EC).

1.2 Scope

The scope is the client application (Web Application, asset code **ASSET-DYN-QA-001**, version 1.0.0). The backend service (`transportBackEndNative`, REST + Socket.IO) is described only at the level of the contracts this SPA depends on; its internal design is outside the scope of this document. The console operates a single kind of tenant — a *loading place* or *Queue Organization* — and provides three jobs: loading-place management, the FIFO driver queue, and bidding on behalf of shippers.

1.3 References

- `src/utils/constant.ts` — single source of truth for REST endpoint paths
- `src/lib/redux/api/*.ts` — endpoint modules and RTK Query contract
- `src/lib/redux/api/base.ts` — base query, credentials, 401 → logout
- `src/lib/socket.ts` — Socket.IO client, subscriptions, debounced invalidation
- `src/routes/AppRoutes.tsx` — route table and lazy loading
- `src/components/auth/ProtectedRoute.tsx`, `src/components/auth/RoleGuard.tsx`
- `src/types/queue.ts`, `src/lib/redux/api/types.ts` — domain types
- `src/data/insaStandard.ts` — INSA standard content for the compliance dashboard
- `vite.config.ts`, `.env.example` — build and environment model
- `docs/insa/assets/*.mmd` and `*.svg` — figure sources

1.4 Definitions

Term	Definition
Queue Organization	A “loading place” tenant of the console (customs, factory, cement, depot, mine, farm, port, other) with coordinates, an optional check-in radius, and an approval status.
Queue Org Admin	Role id 11. Administers the loading place: profile, membership, and approval of bidding per order.
Queue Dispatcher	Role id 12. Runs the FIFO queue: manual check-in, dispatch, override, remove, history.
Admin / Super Admin	Role id 3 / 6. Approve loading places and oversee all organizations. Short session (24h).
FIFO queue	First-in-first-out driver waiting line per vehicle type, keyed by `queueNumber`; entries are dispatched, overridden, or removed.

Term	Definition
RTK Query	Server-state data fetching/caching layer; one base `api` extended by `injectEndpoints`.
Entry history / audit	Per-entry change log (column, old value, performed by, performed at) and the backend queue audit trail.

2. System overview and context

Dynamics Queue Admin is the dispatch-queue management console of the Dynamics freight marketplace. Display name “Queue Admin”. Asset type: Web Application. Asset code **ASSET-DYN-QA-001**. Version 1.0.0. It is a single-page React application served in production at `queue.dynamicsroute.tech`.

A Queue Organization is a physical loading place. Drivers who arrive at the place prove presence by GPS check-in and join a FIFO line; dispatchers assign queued drivers to shipper requests and can override or remove an entry with an audited reason; queue-org admins administer the place’s profile and members and approve bidding per order; platform admins approve origins and oversee all organizations. The backend (`transportBackEndNative`) is the authority for all business rules; the console is a presentation and orchestration layer over its REST and Socket.IO contracts.

2.1 External actors and systems

Actor/System	Role
Shipper	Creates freight requests (individual or batch); receives bid outcomes for dispatched drivers.
Driver / Vehicle Owner	GPS-proven check-in, FIFO queue position, bids, and journey status updates. Represented by a vehicle-driver assignment.
Queue Org Admin (11) / Queue Dispatcher (12)	Primary users of the console; operate the loading-place queue and bidding.
Admin (3) / Super Admin (6)	Approve loading places and oversee organizations across the marketplace.
Backend REST API	`/api` endpoints consumed via RTK Query and legacy axios (`src/utils/constant.ts`).
Backend Socket.IO	Realtime queue fan-out; client subscribes with `queue:subscribe` (`src/lib/socket.ts`).
Photon geocoder (optional)	Place search for order origin/destination (`VITE_PHOTON_URL`, default `photon.komoot.io`). Falls back to a built-in list of Ethiopian cities.
OpenStreetMap / Nominatim / OSRM	Listed as optional external map services; not wired in this console asset (place search uses Photon). Where absent, no map tiles or routing calls are made.

2.2 Level 0 context

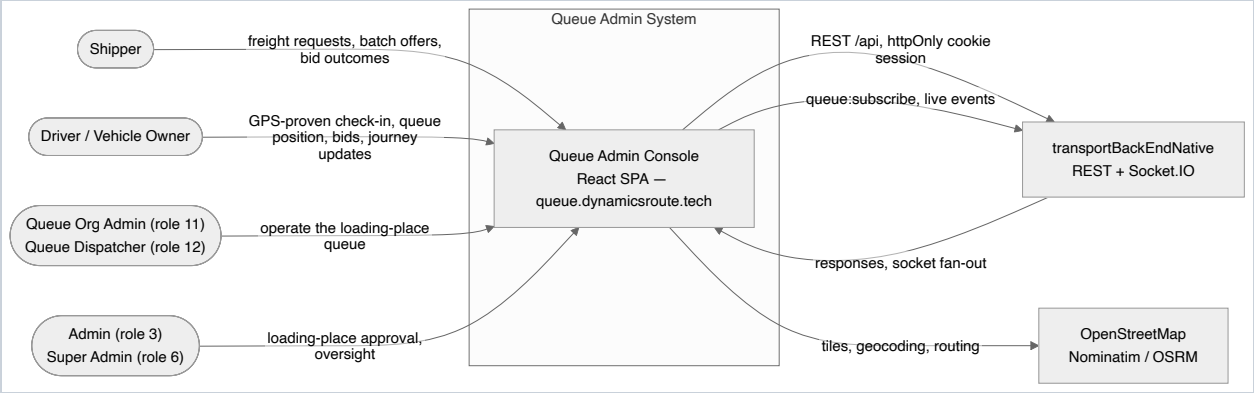


Figure 2.1. Level 0 — Context diagram (external ecosystem). Flows: freight requests / batch offers / bid outcomes, GPS-proven check-in and queue position, loading-place operation, approval and oversight, REST `/api` + cookie session, and Socket.IO events (per `assets/l0.mmd`).

3. Architectural design

3.1 Architectural goals

- Single-page React SPA with strict, layered route authorization (`ProtectedRoute` → `RoleGuard`).
- Centralized API access: one RTK Query base `api` extended by feature endpoint modules, plus a legacy axios instance for a small set of services.
- Cookie-based session with no token in JavaScript; a single 401 handler logs the user out everywhere.
- Realtime synchronization through Socket.IO subscriptions with debounced cache invalidation.
- Clear separation of concerns across presentation, application, state/data, API contract, and external layers.
- Hand-written CSS with design tokens for a lightweight, dependency-free visual system.

3.2 Layered model

Layer	Contents	Responsibility
Presentation	Pages and components under `src/pages/*` and `src/components/*` ; hand-written CSS (`src/index.css` tokens + co-located sheets; `src/dark.css`)	Render UI, queue board, orders, orgs, reports, settings, security; accessibility.
Application	`src/routes/AppRoutes.tsx` , guards, `src/context/AuthContext.tsx` , `src/contexts/ThemeContext.tsx` , composed hooks, forms (RHF + zod)	Routing, authorization, session lifecycle, form state and validation.
State & Data	RTK Query store (`src/lib/redux/store.ts`), slices (`authSlice` , `queueSlice`), zustand (`src/store/queueAdminStore.ts`), Socket.IO client, legacy axios services	Server-state cache, client state, realtime synchronization, HTTP transport.
API contract	`src/utils/constant.ts` and endpoint modules (`authEndpoints` , `queueOrganizationEndpoints` , `driverQueueEndpoints` , `shipperRequestEndpoints` , `vehicleEndpoints`)	Declare endpoints, methods, arguments, responses, and cache tags.
External	Backend REST `/api` , Socket.IO, optional Photon geocoder	Remote systems contracted through the API layer.

3.3 Technology selection

Technology	Version	Justification
Vite	8	Fast dev/build; env-driven base URL, dev proxy, dynamic-import route chunks, manual chunking.
React	19	SPA UI with concurrent rendering; StrictMode in `main.tsx` .
TypeScript	6	Type safety across store, endpoint contracts, components, and domain types.

Technology	Version	Justification
react-router-dom	7 (BrowserRouter)	Client routing, redirects, lazy routes, guarded access.
Redux Toolkit + RTK Query	2	Predictable client state plus server-state caching, tags, and invalidation.
React Context	—	Low-frequency global state: authentication (`AuthContext`) and theme.
zustand	5	Single persisted store for `selectedOrgId` and the socket-connected flag.
react-hook-form + zod	7 / 4	Form state and schema validation with input sanitization helpers.
axios	1	Shared instance for legacy services; `withCredentials` and 401 interceptor.
socket.io-client	4	Realtime queue events; subscriptions and debounced cache invalidation.
sonner	2	User-facing toast notifications for operations and errors.
date-fns	4	Date formatting and comparison for queue/order dates.
i18next / react-i18next	26 / 17	English + Amharic localization with inline fallback strings.
lucide-react	—	Consistent icon set used across the UI.
Hand-written CSS	—	Design tokens in `src/index.css` plus co-located sheets; no Tailwind.

3.4 Level 2 architecture layers

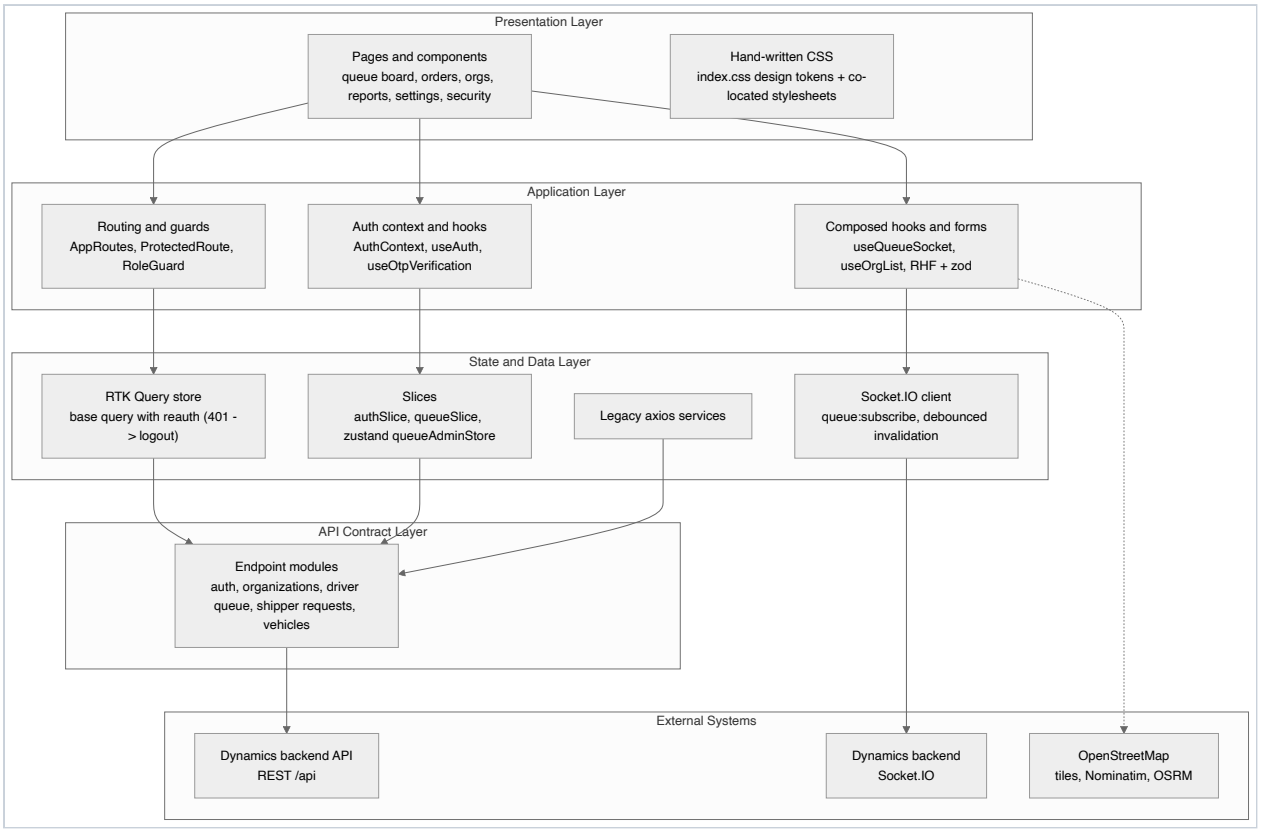


Figure 3.1. Level 2 — Architecture layers (presentation, application, state/data, API contract, external) and data paths (per `assets/l2.mmd`).

3.5 Single-page routing and the guard stack

The application is a single `BrowserRouter` mounted in `src/main.tsx`. Routes are declared in `src/routes/AppRoutes.tsx`; every page is `React.lazy`-loaded and wrapped in a `Suspense` fallback (`PageLoader`), so each route ships in its own dynamic-import chunk. Default redirects send `/` and `*` to `/dashboard`.

Authorization is a two-stage guard stack. `ProtectedRoute` requires an authenticated user (`auth.userData`); by default it also requires at least one queue organization, redirecting to `/setup-org` when none exists (the `/setup-org` route passes `requireOrg={false}`). Nested inside it, `RoleGuard` admits only role ids `{11, 12, 3, 6}` and otherwise redirects to `/login`. Client guards are a usability boundary only; the backend re-checks every request.

4. Functional decomposition

4.1 Level 1 decomposition

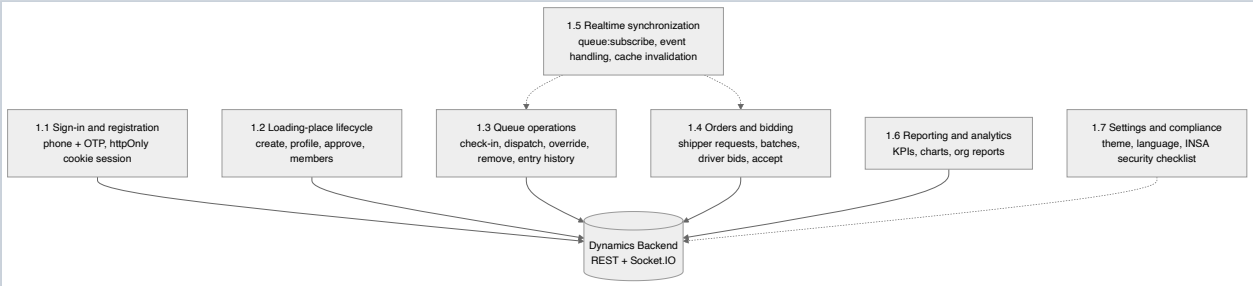


Figure 4.1. Level 1 — Functional decomposition of console capabilities (per `assets/l1.mmd`).

4.2 Key processes

Process	Description	Key modules
Sign-in and session	Phone number → OTP; the backend sets an httpOnly session cookie at `POST /api/user/verifyUserByOTP`. `AuthContext` stores the non-secret profile and connects the socket; logout calls `/api/user/logout` and clears state.	`src/components/auth/*`, `src/hooks/useOtpVerification.ts`, `src/context/AuthContext.tsx`
Loading-place approval	Create/edit the Queue Organization profile and drive the platform approval workflow (pending → approved / rejected / suspended). Admins (3/6) and queue-org admins (11) act through `/queueOrganization/:id/approve`; `queueEnabled` is toggled on approval.	`src/components/organization/SetupOrganization.tsx`, `src/pages/queue-org-manage/QueueOrgManagePage.tsx`, `src/components/queue-org-manage/*`
FIFO queue lifecycle: check-in → queue → dispatch	Operators manually check a vehicle-driver into the line (`/queue/manualCheckin`); the returned `queueNumber` places the driver in FIFO order per vehicle type. Dispatch assigns a queued entry to a shipper request (`/queue/dispatch`).	`src/components/queue/CheckinModal.tsx`, `src/components/queue/checkin/*`, `DispatchModal.tsx`, `QueueBoard.tsx`
Check-in radius / Haversine gate	A Queue Organization carries an optional `checkinRadiusKm`. The console computes great-circle distance with a Haversine implementation (`calculateDistanceKm`) for proximity cues; the authoritative geofence decision for a GPS check-in is enforced server-side.	`src/utils/formatters.ts` (`calculateDistanceKm`), `src/components/orders/bids/BidRow.tsx`
Override / remove with audit	A dispatcher repositions an entry with a reason (`PATCH /queue/entry/:queueUniqueId/override`) or removes it (`DELETE /queue/entry/:queueUniqueId`); both are audited in the backend queue audit trail.	`src/components/queue/OverrideModal.tsx`, `ConfirmCancel.tsx`, `src/lib/redux/api/driverQueueEndpoints.ts`
Entry history	Per-entry change log rendered from `GET /queue/entry/:queueUniqueId/history` (column, old value, performed by, performed at).	`src/components/queue/EntryHistoryModal.tsx`

Process	Description	Key modules
Bidding: approve → list → accept	Approve or close bidding per order (`POST /queue/bidding/approve`), list the authoritative driver bids cheapest-first (`GET /queue/bidding/order/:shipperRequestUniqueId/bids`), and accept a bid using explicit ids via `PUT /shipper/acceptDriverOffer` . A missing id is a hard, visible failure — no fuzzy reconstruction.	`src/components/orders/bids/*` , `src/components/queue/ShipperRequestsModal.tsx` , `DriverBidsModal.tsx`
Realtime subscribe / invalidate	The console subscribes to a Queue Organization room (`queue:subscribe`) and, on events, debounces and invalidates RTK Query tags (100 ms for org events, 250 ms for queue events) to trigger refetches.	`src/lib/socket.ts` , `src/hooks/useQueueSocket.ts`
Reporting	KPIs, donut/bar charts, and per-organization reports computed client-side from organization, queue-status, and shipper-request queries.	`src/pages/reports/ReportsPage.tsx` , `src/hooks/useReportsMetrics.ts` , `src/components/reports/*`
Security compliance dashboard	Renders the INSA Secure Website Management Standard content (focus areas, requirements, vulnerabilities, and a 102-item security test checklist) from `src/data/insaStandard.ts` , with locally persisted pass/fail marks.	`src/pages/security/SecurityPage.tsx` , `src/components/security/*` , `src/hooks/useSecurityChecklist.ts` , `src/utils/securityChecklist.ts`

5. Component design

Routes and guards are defined in `src/routes/AppRoutes.tsx`; guards live in `src/components/auth/`. Pages compose feature components; feature components consume RTK Query hooks from `src/lib/redux/api`.

5.1 Routing and authorization

Route	Component	Guard
`/login`	`Auth` (`initialStep="login"`)	Public
`/register`	`Auth` (`initialStep="register"`)	Public
`/setup-org`	`SetupOrganization`	`ProtectedRoute requireOrg={false}`
`/organizations`	`Navigate → /dashboard`	— (redirect)
`/dashboard`	`QueueDashboardPage`	`ProtectedRoute` + `RoleGuard`
`/orgs/:queueOrganizationUniqueId`	`QueueOrgManagePage`	`ProtectedRoute` + `RoleGuard`
`/orgs/:queueOrganizationUniqueId/orders`	`OrdersPage`	`ProtectedRoute` + `RoleGuard`
`/orders`	`OrdersPage`	`ProtectedRoute` + `RoleGuard`
`/reports`	`ReportsPage`	`ProtectedRoute` + `RoleGuard`
`/security`	`SecurityPage`	`ProtectedRoute` + `RoleGuard`
`/settings`	`SettingsPage`	`ProtectedRoute` + `RoleGuard`
`/`, `*`	`Navigate → /dashboard`	— (redirect)

5.2 Feature areas

Feature	Components / Pages	Store / API
Queue board, table, cards, modals	`src/components/queue/QueueBoard.tsx`, `QueueBoardHeader.tsx`, `QueueVehicleTypeCard.tsx`, `QueueAllDriversCard.tsx`, `QueueCard.tsx`, `table/QueueDesktopTable.tsx`, `table/QueueMobileTable.tsx`, `QueueBoardModals.tsx`, `CheckinModal.tsx`, `DispatchModal.tsx`, `OverrideModal.tsx`, `ConfirmCancel.tsx`, `EntryHistoryModal.tsx`	`useGetQueueStatusQuery`, `useManualCheckinMutation`, `useDispatchQueueMutation`, `useOverrideEntryMutation`, `useRemoveEntryMutation`, `useGetEntryHistoryQuery`

Feature	Components / Pages	Store / API
Orders and bids	`src/pages/orders/OrdersPage.tsx`, `OrdersTable.tsx`, `OrdersCardsList.tsx`, `OrdersMobileCards.tsx`, `orders/bids/*` (`BidRow`, `BidsToolbar`, `BiddingBoardToggle`, `useAcceptDriverBid`), `orders/DriverBidsModal.tsx`	`useGetShipperRequestsQuery`, `useGetShipperRequestBatchesQuery`, `useGetBidsForOrderQuery`, `useApproveBiddingMutation`, `useAcceptDriverRequestMutation`
Queue organization management	`src/pages/queue-org-manage/QueueOrgManagePage.tsx`, `queue-org-manage/OrgProfileForm.tsx`, `MembersTable.tsx`, `AdminApprovalActions.tsx`, `StatusBadge.tsx`, `organization/SetupOrganization.tsx`	`useCreateQueueOrganizationMutation`, `useUpdateQueueOrganizationMutation`, `useApproveQueueOrganizationMutation`, `useListQueueOrgMembersQuery`, member lifecycle mutations
Dashboard	`src/pages/dashboard/QueueDashboardPage.tsx`, `src/components/dashboard/*` (ActiveOrgView, OrgStatusTable, OrgListToolbar, OrgListState, OrgListPagination, OrgApprovalBanner)	`useOrgList`, `useActiveOrg`, `useListQueueOrganizationsQuery`
Reports	`src/pages/reports/ReportsPage.tsx`, `src/components/reports/*` (KpiCards, DonutChart, MonthlyBarChart, OrgReportsList, OrgReportRow)	`useReportsMetrics` over organizations/queue/requests queries
Security compliance	`src/pages/security/SecurityPage.tsx`, `src/components/security/*` (SecurityScoreCards, FocusAreaAccordion, VulnerabilityTable, SecurityChecklist)	`useSecurityChecklist`, `src/data/insaStandard.ts`
Settings and layout	`src/pages/settings/SettingsPage.tsx`, `src/components/settings/*`, `src/components/layout/DashboardLayout.tsx`, `Sidebar.tsx`, `src/components/common/MobileHeader.tsx`	`useTheme`, `useAuth`, `AppRoutes`

6. Data design

6.1 Core entities and key attributes

Entity	Key attributes
User	`userId`, `userUniqueId`, `fullName`, `phoneNumber`, `email`, `roleId` (11, 12, 3, 6). Non-secret profile persisted client-side.
QueueOrganization	`queueOrganizationUniqueId`, name, type enum `customs | factory | cement | depot | mine | farm | port | other`, `latitude` / `longitude`, `checkinRadiusKm`, `approvalStatus` (`pending | approved | rejected | suspended`), `queueEnabled`, `approvedBy` / `approvedAt`.
QueueOrganizationMembership	`queueOrganizationMembershipUniqueId`, `userUniqueId`, `roleId` (11/12), `isActive`, `membershipStartDate`. Membership is the permission.
DriverQueue	`queueUniqueId`, `queueNumber`, `queueStatus` (`waiting | offered | loaded | removed | completed`), `queueRefusalCount`, `targetedShipperUserUUID`, `vehicleDriverUniqueId`, `shipperRequestUniqueId`, timestamps.
DriverQueueHistory	`historyUniqueId`, `columnName`, `oldValue`, `performedBy`, `performedAt`.
QueueAuditLog	Backend audit trail for queue mutations (check-in, dispatch, override, remove), including the override reason and the acting user; surfaced in the entry history view.
ShipperRequest	`shipperRequestUniqueId`, `shipperRequestBatchUniqueId`, `requestMode` (`individual_target | company_target`), origin/destination, `shippingCost`, `journeyStatusId`, `queueOrganizationUniqueId`, `isBiddingApproved`.
ShipperRequestBatch	`batchUniqueId`, `totalVehicles`, `requestMode`, origin/destination, `bidSummary`, aggregated bids (fetched with `includeBids=true`, `requestMode=company_target`).
DriverBid	`driverBidUniqueId`, `shipperRequestUniqueId`, `driverUserUniqueId`, `bidAmount`, `bidNotes`, `bidStatus` (`pending | selected | not_selected | withdrawn | expired`), `driverBidCreatedAt`.

6.2 Client state and storage

Store/Storage	Keys/Areas	Notes
RTK Query cache	`api` reducer with tags QueueOrganizations, QueueOrgMembers, QueueStatus, DriverQueue, DriverBids, DriverDirectory, Auth, VehicleTypes, ShipperRequests	`keepUnusedDataFor: 300`; tag-based invalidation; no focus/reconnect refetch.
Redux slices	`authSlice` (auth profile + hydration), `queueSlice` (`selectedOrgId`, `orgsCache`)	Registered in `src/lib/redux/store.ts`.
zustand store	`queueadmin:store` — persisted `selectedOrgId` plus `socketConnected` (transient)	`src/store/queueAdminStore.ts`.

Store/Storage	Keys/Areas	Notes
localStorage	`queueadmin:user` (non-secret profile), `app_dark_mode`, `app_language`	Never stores tokens; legacy `queueadmin:auth` token record is purged on read.
httpOnly cookie	Session cookie (backend-set)	Set at `POST /api/user/verifyUserByOTP`; sent with `credentials: "include"`.

6.3 Data flow (high level)

UI actions invoke RTK Query hooks/mutations or legacy axios services. The base query resolves the API prefix from `VITE_API_BASE_URL` / `VITE_API_URL` (normalized to end in `/api`) and sends the httpOnly cookie via `credentials: "include"`. Responses populate the RTK Query cache; socket events (queue and organization) trigger debounced tag invalidation that refetches the affected queries. Client state (auth profile, selected organization, theme, language, socket flag) lives in Redux slices, zustand, React Context, and localStorage; no token is ever placed in JavaScript-accessible storage.

7. Interface design

7.1 REST API

Endpoint paths are centralized in `src/utils/constant.ts` (the single source of truth); the API base resolves to `<VITE_API_BASE_URL>/api`. Methods below reflect the injected endpoint modules. Frontend-callable surface:

Method + path (`/api` prefix)	Purpose	Module
`POST /user/loginUser`	Request login OTP for a phone number.	auth
`POST /user/verifyUserByOTP`	Verify OTP; backend sets the session cookie.	auth
`POST /user/createUser`	Register a queue-org admin account.	auth
`POST /user/logout`	Clear the server session cookie.	auth
`GET /user/getShipperRequest4allOrSingleUser`	List shipper requests (all/single, scoped to the org).	shipper requests
`GET /queueOrganization`	List queue organizations for the user.	queue organization
`POST /queueOrganization`	Create a loading place (approvalStatus pending).	queue organization
`GET /queueOrganization/:id`	Get one organization.	queue organization
`PATCH /queueOrganization/:id`	Update the organization profile.	queue organization
`DELETE /queueOrganization/:id`	Delete (soft) the organization.	queue organization
`PATCH /queueOrganization/:id/approve`	Approve / reject / suspend; toggles `queueEnabled`.	queue organization
`GET /queueOrganization/:id/members`	List members (role 11/12, active flag).	queue organization
`POST /queueOrganization/:id/members`	Add a member (`userUniqueId` in body).	queue organization

Method + path (`/api` prefix)	Purpose	Module
`PATCH /queueOrganization/:id/members/:membershipId/deactivate`	Deactivate membership (revokes access immediately).	queue organization
`PATCH /queueOrganization/:id/members/:membershipId/reactivate`	Reactivate membership.	queue organization
`DELETE /queueOrganization/:id/members/:membershipId`	Remove membership.	queue organization
`GET /queue/status`	Queue snapshot by vehicle type for a date.	driver queue
`POST /queue/manualCheckin`	Check a vehicle-driver in; returns `queueNumber`.	driver queue
`POST /queue/dispatch`	Dispatch a queued entry to a shipper request.	driver queue
`DELETE /queue/entry/:queueUniqueId`	Remove a queue entry (audited).	driver queue
`PATCH /queue/entry/:queueUniqueId/override`	Override position with a reason (audited).	driver queue
`GET /queue/entry/:queueUniqueId/history`	Entry change history.	driver queue
`GET /queue/driverDirectory`	Search active driver/vehicle assignments for manual check-in (queue staff only).	vehicles
`POST /queue/bidding/approve`	Open/close bidding per order (`shipperRequestUniqueIds`, `approved`).	driver queue / bids
`GET /queue/bidding/order/:shipperRequestUniqueId/bids`	Authoritative driver bids for an order (cheapest-first).	driver queue / bids
`POST /shipperRequest/createRequest`	Create a shipper request / batch order.	shipper requests
`GET /shipperRequestBatch`	List batches (`company_target`, `includeBids`).	shipper requests
`PUT /shipper/acceptDriverOffer`	Accept a driver bid with explicit ids.	driver queue / bids
`GET /admin/vehicleTypes`	Reference vehicle types (falls back to built-in defaults).	vehicles

7.2 Authentication / session contract

- Primary session is an httpOnly cookie set by the backend at `POST /api/user/verifyUserByOTP`. The client never reads or stores a JWT (`getToken()` returns `null`).
- Every request sends the cookie with `credentials: "include"` (`src/lib/redux/api/base.ts`, `src/lib/api.ts`). No `Authorization` header is set.
- There is no refresh-token flow: any `401` response in the RTK Query base query dispatches `logout()` (`baseQueryWithReauth`); the axios instance clears auth and routes to `/login`.
- Session lifetime is role-based on the backend: roles 3/6 = 24h; role 11/12 and others = 365d.

7.3 Socket.IO protocol

The client connects with `withCredentials` and an `auth` payload of `{ user, phoneNumber }`. The socket identity is derived from the account role: roles 3/6 map to `admin`; queue staff (roles 11 and 12) share a single `queueOrgAdmin` identity, because the backend fan-out keys queue events under that identity for both roles. Subscriptions use `queue:subscribe` / `queue:unsubscribe` with a reference-counted room keyed by `queueOrganizationUniqueId` (and optional `queueDate`); rooms are re-subscribed automatically on (re)connect. Inbound queue events are parsed for a `messageTypes` value; acknowledgements and ping/pong are ignored; organization events invalidate `QueueOrganizations` after a 100 ms debounce, and all other queue events invalidate `QueueStatus`, `DriverQueue`, and `ShipperRequests` after a 250 ms debounce. A socket `connect_error` whose message indicates `unauthorized` / `token` disconnects and ends the session.

7.4 Third-party services

Place search for order origin/destination uses the Photon geocoder (`VITE_PHOTON_URL`, default `https://photon.komoot.io/api/`), merged with a built-in list of Ethiopian cities and with a local fallback if the external service is unavailable. OpenStreetMap / Nominatim / OSRM are noted as optional external map services and are not wired into this console asset.

7.5 Error-handling contract

`src/utils/parseError.ts` normalizes axios errors, RTK Query errors, serialized errors, and generic throws into a single human-readable message; it reads server `message` / `error` / `details` / `validation` fields and maps HTTP status codes (400/401/403/404/409/429/5xx) to fallbacks. UI call sites surface the result through `sonner` toasts. Status semantics are distinguished: `401` means “session expired — log in again” and triggers logout; `403` means “authenticated but not permitted” and is shown as a permission error without logging out.

8. Security design

8.1 Authentication and authorization

- OTP login establishes an httpOnly cookie session; the client holds only the non-secret profile. No JWT is stored or transmitted from JavaScript.
- Route access is guarded by `ProtectedRoute` (authentication + organization presence) and `RoleGuard` (roles `{11, 12, 3, 6}`). This is a usability boundary; the backend independently enforces role and membership on every request.
- A single `401` handler in the base query logs the user out globally; a socket `401`/unauthorized handshake also ends the session and stops reconnect attempts.

8.2 Cookie session and CSRF posture

Because the session is an httpOnly cookie sent with `credentials: "include"`, the CSRF posture depends on backend controls (same-site cookie attributes and trusted-origin checks) that are outside this client asset. The client does not add custom CSRF tokens and does not store credentials in script-accessible storage. The console never sets `document.cookie`.

8.3 Role enforcement boundary (client vs backend)

Concern	Client	Backend (authority)
Route visibility	Guards hide/redirect unauthorized routes.	Re-checks role/membership per request.
Bidding approval	Toggle shown only to role 11 (queue-org admin).	Enforces approval permission server-side per order.
Membership = permission	Reflects active membership in the members table.	Requires an active membership row (role 11/12) on each request.
Check-in radius	Computes Haversine distance for proximity cues.	Authoritative geofence decision for GPS check-in.

8.4 Input validation and sanitization

Forms use `react-hook-form` with `zod` schemas (`src/schemas/queue.ts`) covering login, registration, organization setup and profile, check-in, dispatch, order creation, and override. Schemas constrain types, lengths, enums, UUIDs, coordinate ranges, and quantity limits, and include sanitizers that strip HTML tags, control characters, and collapse whitespace (`sanitizeText`, `sanitizeEmail`). React escapes rendered values by default, and i18n is configured with `escapeValue: false` because React performs escaping.

8.5 Storage hygiene and build hardening

- localStorage holds only non-sensitive values (`queueadmin:user` profile, `app_dark_mode`, `app_language`, persisted `selectedOrgId`); tokens are never stored, and legacy token records are purged on read.
- Production source maps are disabled (`sourcemap: process.env.NODE_ENV !== "production"`).

- **Honest note:** this repository does not configure a Content Security Policy or other HTTP security headers. Header/CSP hardening is a server-level responsibility of the hosting environment, which is outside the scope of this code asset; the gap is recorded in §13 and treated as deferred/mitigated at the platform boundary.

8.6 INSA compliance dashboard

The console includes a security-compliance dashboard that encodes the INSA Secure Website Management Standard (Version 1.0, 2014 EC) as a self-assessment. Content is sourced from `src/data/insaStandard.ts`: **5 focus areas** (requirement gathering, design, implementation, hosting, operation & management), **35 requirements**, **8 common vulnerabilities** (Annex A), and a **102-item security test** checklist (Annex B). The checklist lets a reviewer mark each test pass/fail, with results persisted locally (`useSecurityChecklist`) and scored by `src/utils/securityChecklist.ts`.

9. Deployment and hosting

9.1 Build

Static build via Vite (`npm run build` = `tsc -b && vite build`), output to `dist/`. The Rollup output splits vendor libraries: a `vendor` chunk (react, react-dom, react-router-dom), a `redux` chunk (@reduxjs, react-redux, zustand), and an `icons` chunk (lucide-react). Route components are separate dynamic-import chunks.

9.2 Environment and API base resolution

The API base is resolved at compile time from `VITE_API_BASE_URL`, falling back to `VITE_API_URL` and then to `/api`; the value is normalized to end in `/api` (`src/utils/baseUrl.ts`). In development, Vite proxies both `/api` and `/socket.io` (with WebSocket upgrade) to `VITE_API_BASE_URL` when set. The socket URL prefers `VITE_WEBSOCKET_URL` / `VITE_SOCKET_URL`, falling back to the API base or the page origin.

9.3 Hosting and SPA fallback

The built `dist/` is served as a static site over TLS at `queue.dynamicsroute.tech`. Because routing is client-side, the host must serve the SPA entry (`index.html`) for unmatched paths so deep links such as `/orgs/:id` resolve; the in-app `*` route then redirects to `/dashboard`. Session cookies must be delivered over HTTPS with backend-set attributes.

9.4 Environments

Env	SPA	API
Production	`https://queue.dynamicsroute.tech`	`https://app.dynamicsroute.tech`
Development	local dev (Vite, port 5173)	`https://dev.dynamicsroute.tech`

10. Cross-cutting design

- **Internationalization:** i18next/react-i18next with English and Amharic resources (`src/i18n/en.ts`, `src/i18n/am.ts`); components use inline fallback strings, and the language persists in `app_language`.
- **Theme:** light/dark mode via `ThemeContext` (`src/contexts/ThemeContext.tsx`), toggling the `dark` class on the document root and applying `src/dark.css`; preference persists in `app_dark_mode`.
- **Responsive/mobile:** dedicated mobile views and headers (`MobileHeader`, mobile queue/order cards, mobile settings) alongside desktop layouts, driven by hand-written CSS breakpoints.
- **Accessibility:** semantic elements, ARIA roles/labels on interactive controls and modals (`useModalA11y`), keyboard handling (OTP inputs, custom selects, sidebar items), and status regions.
- **Toasts:** `sonner` renders operation and error feedback at the application root.
- **Date handling:** `date-fns` plus shared formatters for queue/order dates.

> **Error boundaries**
>
> This console does not currently declare a React error-boundary component; uncaught render errors are not contained by an in-app boundary. This is recorded as a known limitation (§13).

11. Design decisions and trade-offs

Decision	Rationale	Trade-off
RTK Query over react-query	One store, tag-based cache invalidation, and a single 401 reauth slot fit the queue's shared cache.	`@tanstack/react-query` remains a dead dependency and should be removed.
React Context for auth and theme	Small, low-frequency global state; avoids extra Redux plumbing.	Context re-renders; unsuitable for high-frequency data (that lives in RTK Query).
Cookie session over bearer token	Keeps the JWT out of JavaScript and removes token-handling code.	Depends on backend cookie/CSRF controls; cross-site requests need `credentials: "include"`.
Hand-written CSS	Full control, no build/utility dependency, small footprint.	More manual consistency work than a design system or utility framework.
Single socket identity for both queue roles	The backend fans queue events to the `queueOrgAdmin` key for roles 11 and 12; a separate dispatcher identity would silently receive nothing.	Client cannot distinguish role by socket identity; role checks stay per-request.
Debounced full refreshes over per-event patching	Event payloads vary and can be partial; invalidating authoritative tags is simpler and safer.	Extra network round-trips; a burst of events coalesces into one refetch.
Accept bids by explicit ids	Prevents the race where fuzzy id reconstruction silently accepts a different driver.	A missing id is a hard error the operator must resolve by reloading bids.

12. Requirements-to-design traceability

Traceability maps SRS requirement families (FR-*/NFR-*) to design elements. Key mappings:

SRS family	Design reference	Notes
FR-AUTH	`useOtpVerification`, `AuthContext`, `services/auth.service.ts`, `base.ts`	OTP login, cookie session, global 401 logout, socket 401 session end.
FR-ROLE/ACCESS	`ProtectedRoute.tsx`, `RoleGuard.tsx`	Roles {11, 12, 3, 6}; organization-presence requirement.
FR-ORG	`SetupOrganization`, `QueueOrgManagePage`, `OrgProfileForm`, membership mutations	Create/edit profile, approval workflow, membership lifecycle.
FR-QUEUE	`QueueBoard` and queue components, `driverQueueEndpoints.ts`	Check-in → queue → dispatch, override/remove, entry history, FIFO number.
FR-BID	`src/components/orders/bids/*`, `DriverBidsModal`, `approveBidding` / `getBidsForOrder` / `acceptDriverRequest`	Approve → list → accept via explicit ids.
FR-ORDER	`OrdersPage`, `CreateOrderModal`, shipper request endpoints	List/view shipper requests and batches; create request.
FR-REALTIME	`src/lib/socket.ts`, `useQueueSocket`	Subscribe/unsubscribe and debounced tag invalidation.
FR-REPORT	`ReportsPage`, `useReportsMetrics`	Client-side KPIs, charts, and per-org reports.
NFR-SEC	`parseError`, zod schemas, `lib/auth.ts`, `src/data/insaStandard.ts`	Input validation, storage hygiene, INSA compliance self-assessment.
NFR-DEPLOY	`vite.config.ts`, `baseUrl.ts`, hosting at `queue.dynamicsroute.tech`	Env-driven base URL, dev proxy, static SPA fallback.

13. Known limitations and future work

Item	Status	Planned action
No refresh-token flow	NOTED	Any expiration ends the session (401 → logout); add refresh only if the backend offers it.
Settings: profile / notifications / 2FA disabled	NOTED	No profile PATCH and no notification/2FA backend; controls are intentionally disabled rather than faked.
Dashboard counts computed client-side	NOTED	Metrics are derived from fetched lists (limit 100) in the browser; move to server aggregates for scale.
Socket payloads trigger refetch only	BY DESIGN	Events invalidate tags; no per-event cache patching is applied.
Dead `@tanstack/react-query` dependency	OPEN	Nothing imports it; remove from `package.json`.
Unreachable `/organizations` page	OPEN	The route redirects to `/dashboard`; the standalone page is not reachable and should be deleted or re-wired.
No CSP or HTTP security headers in this repo	DEFERRED	Server-level control, outside this code asset; mitigate at hosting/reverse-proxy layer.
Optional third-party map use	OPTIONAL	Only Photon place search is wired; OSM/Nominatim/OSRM remain optional.
No in-app error boundary	OPEN	Add a top-level React error boundary to contain render errors.

Placeholders

TBD is used only where strictly unknown (submission entity, contact, prepared/reviewed by, signatories, date). All technical details above reflect the actual repository state.